软件设计说明书（SDD）

智能在线编程练习平台

编制人：冯朝航

修订日期：2026 年 04 月 08 日

修订历史

修订人	修订日期	变更原因	版本号
冯朝航	2026.04.08	文档首次编制，完成全模块架构与设计说明	V0.1

1. 引言

1.1 文档目的

本软件设计说明书（SDD）是《智能在线编程练习平台需求分析文档》的配套技术文档，完整定义了平台的架构设计、模块划分、核心实现逻辑、接口规范、数据结构与部署方案。

本文档面向项目研发人员、测试人员、项目管理人员、指导老师与评审人员，核心作用如下：

1. 为平台前后端开发、模块集成、测试验证提供统一的技术基准与实现规范；
2. 明确系统架构与模块边界，保障团队协同开发的一致性，降低模块耦合度；
3. 作为项目进度管控、质量验收、后续迭代维护的核心技术依据；
4. 完整对齐需求分析文档的所有功能与非功能要求，确保设计方案可落地、可验证。

1.2 项目范围

1.2.1 系统定位

本系统是基于 SpringBoot 框架开发的 Web 端智能编程练习平台，采用“传统 OJ 判题引擎+AI 大模型智能赋能”双引擎模式，为在校学生、编程算法爱好者、求职备考人群提供一站式算法学习解决方案，同时为管理员提供完整的平台运营管理能力。

1.2.2 包含范围

1. 网页端产品：面向普通用户的刷题系统、面向管理员的后台管理系统；
2. 核心功能模块：用户管理模块、题库管理模块、在线编程与自动判分模块、AI 智能辅助模块、数据统计模块、后台管理模块；
3. 底层能力支撑：多语言在线代码编辑器、C++自研 OJ 评测引擎、Docker 安全沙箱运行环境、AI 大模型接口对接能力；
4. 全流程交付：平台开发、集成测试、性能与安全测试、部署上线、全套项目文档交付。

1.2.3 不包含范围

1. 移动端 APP、微信小程序等移动端产品开发；
2. 多人在线编程竞赛、实时对战、直播教学等互动功能；
3. 付费会员、在线支付、商业变现相关功能；
4. 多服务器分布式集群部署、高可用商业化运维方案；
5. 用户社区、评论互动、内容分享相关功能。

1.3 定义、首字母缩写词和缩略语

术语	完整名称	定义
API	应用程序编程接口	应用系统之间、模块之间数据交互与功能调用的规范集合
SDD	软件设计说明书	描述软件设计目标、架构方案、模块逻辑、实现规范与部署方案的核心技术文档
OJ	在线判题系统	本项目中实现用户代码在

		线编译、隔离运行、自动判分的核心引擎模块
AI	人工智能	本项目中用于代码评价、题目生成、智能解析、编程答疑的大模型服务
B/S	浏览器/服务器架构	本项目采用的软件架构模式，用户通过浏览器即可访问平台，无需安装客户端
SpringBoot	后端开发框架	本项目后端采用的 Java 开源开发框架，用于快速搭建企业级应用服务
MySQL	关系型数据库	本项目采用的结构化数据存储数据库，用于存储平台全量业务数据
Docker	容器化引擎	本项目用于构建代码运行安全沙箱的容器化技术，实现运行环境隔离
JWT	JSON Web Token	本项目用于用户身份认证的令牌技术，实现无状态的用户登录状态管理
WebSocket	全双工通信协议	本项目用于实时推送判分进度、AI 生成进度的通信协议
POC	概念验证	项目启动前对核心技术点进行的可行性验证测试

1.5 文档概述

1. 引言：明确文档目的、项目范围、术语定义、参考资料与文档结构；
2. 设计概述：从多维度视角描述系统整体设计思路，明确涉众核心诉求与设计覆盖范围；
3. 设计视图：详细阐述系统整体架构、各模块详细设计、数据库设计、接口设计、安全与性能设计，是本文档的核心内容；
4. 关键设计决策：明确项目核心架构、技术选型、实现方案的决策内容与选型依据；
5. 附录：补充项目配套文档、开发规范、部署环境等支撑性材料。

本文档所有设计方案 100%对齐需求分析文档的功能与非功能要求，所有技术选型均匹配项目团队能力与 12 周开发周期要求，确保方案可落地、可验证、可交付。

2. 设计概述

2.1 涉众关注点

本文档覆盖所有项目涉众的核心设计诉求，各涉众的核心关注点如下：

涉众角色	核心设计关注点
普通用户	1. 页面响应速度、操作流畅度，代码编辑器无卡顿；2. 代码判分结果 100%准确，反馈及时；3. AI 功能可用、生成内容准确合规；4. 个人数据安全，账号信息不泄露；5. 核心刷题流程操作便捷，新手可快速上手；6. 平台运行稳定，无崩溃、数据丢失问题
平台管理员	1. 后台管理功能完整，覆盖题库、用户、内容、平台配置全流程；2. 权限管控严格，越权操作 100%拦截；3. AI 生成内容审核流程完善，违规内容可快速处置；4. 平台运行状态可实时监控，异常问题可快速定位；5. 运营数据可视化展示，统计维度完整
后端开发人员	1. 架构分层清晰，模块职责单一，低耦合高内聚；2. 接口规范统一，入参出参格式标准；3. 核心业务流程逻辑明确，异常处理机制完善；4. 代码可扩展性强，预留后续功能迭代空间；5. 技术选型匹配团队能力，无过度设计

前端开发人员	1. 前后端接口定义清晰，字段含义明确；2. 实时数据推送方案成熟，无兼容性问题；3. 代码编辑器选型适配项目需求，二次开发便捷；4. 页面交互逻辑与需求完全对齐，状态管理方案清晰
测试人员	1. 功能模块边界清晰，测试点可全覆盖；2. 异常处理逻辑明确，可针对性设计测试用例；3. 性能与安全设计有明确指标，可量化验证；4. 核心流程有明确的状态流转规则，可追溯测试
项目负责人	1. 设计方案完全覆盖需求，无功能遗漏；2. 技术方案风险可控，核心难点有成熟的解决方案；3. 开发工作量可拆解，匹配 12 周项目周期；4. 方案符合毕业设计/立项答辩的评审要求
项目指导老师	1. 架构设计合理，技术选型符合行业规范；2. 核心技术点有创新性，匹配项目定位；3. 设计方案考虑了安全性、可扩展性、可维护性；4. 项目风险有对应的设计层面应对措施

2.2 选定的设计视角

2.2.1 上下文视角

本视角明确系统的业务边界、外部交互实体与核心交互关系，界定系统能力范围。

1. **系统核心边界**：系统核心为智能在线编程练习平台的前后端服务，核心能力为编程题目管理、在线代码编写、自动判分、AI 智能辅助、用户与权限管理，不超出需求定义的功能范围。
2. **外部交互实体**：
    - 内部实体：普通用户、平台管理员、超级管理员；
    - 第三方服务实体：AI 大模型服务、邮件验证码服务、云服务器基础设施；
    - 存储实体：MySQL 数据库、本地文件存储系统。
3. **核心交互关系**：
    - 用户与系统：通过浏览器完成页面访问、功能操作、数据提交；
    - 系统与第三方服务：通过 API 接口完成 AI 能力调用、邮件验证码发送；

- 系统与存储实体：通过数据访问层完成业务数据的持久化存储与读取。

2.2.2 组成视角

本视角明确系统的核心模块组成、模块职责与组织关系，系统整体分为 6 大核心功能模块，各模块独立封装，通过标准化接口完成交互。

模块名称	核心职责	下属子模块
用户管理模块	负责用户全生命周期管理，保障账号安全与用户数据管理	账号注册登录、个人信息管理、密码管理、提交记录管理、收藏管理、错题本管理、账号注销
题库管理模块	负责题目与分类的全生命周期管理，为前端提供题目检索与展示能力	题目分类管理、题目基础信息管理、测试用例管理、官方题解管理、题目检索与展示
在线编程与自动判分模块	负责在线代码编辑、代码提交与自动判分核心逻辑，是平台的核心引擎	在线代码编辑器、代码提交预处理、自动判分核心引擎、判分沙箱管理、判分结果推送与展示
AI 智能辅助模块	负责对接 AI 大模型，实现所有 AI 相关的智能能力	AI 代码评价、AI 题解生成、AI 题目生成、AI 编程答疑、内容合规校验
数据统计模块	负责用户学习数据与平台运营数据的统计、分析与可视化展示	个人学习数据统计、平台运营数据统计、数据可视化、学习建议生成
后台管理模块	负责平台运营管理、权限管控与运行监控	用户与权限管理、AI 内容审核管理、平台配置管理、运行状态监控、操作日志管理

2.2.3 逻辑视角

本视角明确系统的分层架构设计，采用经典的五层分层架构，实现职责分离、层级解耦，每层仅与相邻层级交互，无跨层依赖，保障架构的清晰性与可维护性。

架构层级	层级职责	核心实现内容

表现层	负责用户交互与页面渲染，接收用户操作请求，展示处理结果	用户端 Web 页面、管理员后台页面、在线代码编辑器、数据可视化组件
接入层	负责请求转发、流量管控、安全过滤与跨域处理	Nginx 反向代理、请求限流、跨域配置、静态资源服务、HTTPS 加密
应用服务层	负责核心业务逻辑实现，接收接入层请求，调用基础设施层能力完成业务处理	六大核心功能模块的业务服务、权限校验、事务管理、异常处理
基础设施层	为应用服务层提供底层技术能力支撑，封装第三方服务与基础组件	OJ 判分引擎、Docker 沙箱管理、AI 服务客户端、邮件服务客户端、数据访问组件、缓存组件
数据存储层	负责平台全量数据的持久化存储与读取	MySQL 关系型数据库、本地文件存储系统

2.2.4 物理视角

本视角明确系统运行的硬件拓扑与基础设施约束，匹配项目单服务器部署的范围要求，无分布式集群设计。

1. **硬件拓扑结构**：采用单台云服务器作为核心硬件载体，所有服务组件均部署于同一台服务器内，无跨服务器节点交互。
2. **服务器配置要求**：2 核 4G 及以上 CPU/内存、100G 及以上硬盘空间、公网带宽 5M 及以上，操作系统为 Linux CentOS 7.9。
3. **基础设施约束**：
    - 服务器需开放 80/443 端口用于 Web 访问，禁止其他非必要端口对外暴露；
    - 服务器需支持 Docker 容器引擎安装，满足沙箱运行环境要求；
    - 服务器需具备公网访问能力，可正常调用第三方 AI 服务与邮件服务；
    - 暂不考虑多服务器冗余部署、灾备集群等商业化运维方案。

2.2.5 结构视角

本视角明确系统的整体组织结构，采用**前后端分离的 B/S 架构**，核心结构分为前端单页应用、后端 RESTful API 服务两大主体，通过标准化接口完成数据交互。

1. **前端结构**：采用单页应用（SPA）架构，分为页面层、组件层、状态管理层、接口请求层，实现页面渲染、交互逻辑与数据请求的解耦；
2. **后端结构**：采用微内核插件化结构，核心框架为 SpringBoot，各业务模块以插件形式集成于核心框架内，模块之间通过内部接口交互，新增模块无需修改核心框架代码；
3. **核心结构特点**：前后端完全解耦，可并行开发；业务模块低耦合，可独立测试、独立迭代；核心引擎（OJ 判分、AI 服务）可独立替换，不影响上层业务逻辑。

2.2.6 依赖视角

本视角明确系统各模块、各层级之间的依赖关系，遵循“单向依赖、弱依赖解耦、核心能力无强依赖”的设计原则。

1. **层级依赖关系**：采用自上而下的单向依赖，表现层→接入层→应用服务层→基础设施层→数据存储层，无反向依赖、无跨层依赖；
2. **模块依赖关系**：
    - 强依赖：题库管理模块→用户管理模块（需校验管理员权限）、自动判分模块→题库管理模块（需获取题目测试用例）、AI 智能辅助模块→题库管理模块+自动判分模块（需获取题目与代码判分数据）；
    - 弱依赖：数据统计模块→其他所有业务模块（仅读取业务数据，不影响核心业务流程）、后台管理模块→其他所有业务模块（仅管理配置，不影响核心业务执行）；
    - 解耦设计：AI 智能辅助模块与核心业务流程解耦，AI 服务故障不影响用户注册登录、代码编写、自动判分等核心功能使用；自动判分模块独立封装，可单独部署、单独升级，不影响其他业务模块。
3. **第三方依赖关系**：系统对 AI 服务、邮件服务均采用弱依赖，配置备用渠道，主渠道故障时可快速切换，不导致对应功能完全不可用。

2.2.7 信息视角

本视角明确系统的核心数据实体、数据结构、数据流转与持久化管理方案。

1. **核心数据实体**：用户信息实体、角色权限实体、题目信息实体、测试用例实体、代码提交记录实体、题目收藏/错题本实体、AI 生成内容实体、操作日志实体、统计数据实体；
2. **数据流转规则**：所有数据均由用户操作或系统自动触发产生，经应用服务层处理

后，通过基础设施层的数访问组件持久化至数据存储层；数据读取遵循“权限管控”原则，普通用户仅可读取本人数据，管理员按角色权限读取对应范围数据；

3. **数据持久化方案**：
    - 结构化业务数据：全部存储于 MySQL 数据库，建立完善的表关系与索引，保障查询性能；
    - 非结构化数据：用户头像、代码草稿、临时文件等存储于服务器本地文件系统，数据库仅存储文件访问路径；
    - 缓存数据：验证码、登录状态、高频访问的题目数据存储于本地缓存，提升访问速度；
4. **数据生命周期管理**：用户账号注销后，按合规要求处理个人敏感数据；用户提交的代码记录、操作日志保留不少于 1 年；平台数据定期备份，备份数据保留周期不少于 30 天。

2.2.8 接口视角

本视角明确系统内外部接口的设计规范、契约与分类，所有接口遵循统一的设计原则，保障交互的标准化与稳定性。

1. **接口分类与设计规范**：

接口类型	通信协议	设计规范	核心用途
前端业务接口	HTTPS RESTful	统一请求/响应格式，标准 HTTP 方法（GET/POST/PUT/DELETE），入参出参 JSON 格式，统一异常处理	常规业务操作，如用户登录、题目查询、信息修改等
实时推送接口	WebSocket	全双工通信，基于会话 ID 绑定用户，心跳检测保活，统一消息格式	判分进度、判分结果、AI 生成进度的实时推送
内部模块接口	Java 本地接口	面向接口编程，统一入参出参规范，异常向上抛出机制	后端各模块之间的内部功能调用

第三方服务接口	HTTPS	适配第三方 API 规范，封装统一客户端，超时重试机制，异常熔断处理	AI 大模型调用、邮件验证码发送

2. **接口设计核心原则**：
   - 接口职责单一，一个接口仅实现一个业务功能；
   - 入参出参字段含义明确，必填/非必填边界清晰，无歧义；
   - 所有接口均有权限校验机制，越权请求 100%拦截；
   - 接口异常处理完善，返回清晰的错误码与错误信息，无未捕获异常；
   - 敏感接口设置限流规则，防止恶意请求刷接口。

2.2.9 交互视角

本视角明确系统核心业务场景的运行时消息流转与组件协作流程，覆盖三大核心业务流程。

1. **用户代码提交与自动判分交互流程**：
   a. 用户前端提交代码→后端接口接收请求，完成身份校验与代码预处理；
   b. 后端服务生成唯一提交编号，将判分任务推入异步任务队列，向前端返回提交成功状态；
   c. 判分调度器从队列中取出任务，调用 Docker 沙箱管理组件创建独立运行环境；
   d. 判分引擎完成代码编译、测试用例逐条执行、结果比对、异常捕获；
   e. 单条用例执行完成后，通过 WebSocket 向用户前端实时推送执行进度；
   f. 全量用例执行完成后，判分引擎汇总最终判分结果，销毁沙箱环境；
   g. 后端服务将判分结果持久化至数据库，通过 WebSocket 向用户前端推送最终判分结果。
2. **AI 功能调用交互流程**：
   a. 用户前端触发 AI 功能请求→后端接口接收请求，完成身份校验、频次限流与参数校验；
   b. 后端服务按 AI 功能类型封装请求参数，调用统一的 AI 服务客户端；
   c. AI 服务客户端通过 HTTPS 接口调用第三方 AI 大模型服务，处理超时与异常情况；

d. 接收到 AI 返回结果后，后端服务完成内容合规校验、数据解析与持久化；

e. 后端服务将最终结果返回给前端页面，完成交互流程。

3. **管理员后台核心交互流程**：

a. 管理员提交操作请求→后端接口完成身份校验与权限校验，拦截越权操作；

b. 后端服务执行对应业务逻辑，完成数据修改/查询；

c. 记录管理员操作日志，将操作结果返回给后台页面，完成交互流程。

2.2.10 算法视角

本视角明确系统核心业务的处理逻辑、关键算法与计算规则，保障核心功能的准确性与高效性。

1. **代码判分核心算法**：

- 结果比对算法：采用标准化输出比对，忽略行尾空格、换行符差异，精准匹配程序输出与标准用例输出；
- 测试用例调度算法：采用串行执行模式，单条用例执行完成后再执行下一条，某条用例超时/异常不影响后续用例执行；
- 运行资源管控算法：基于 Docker 容器的 CPU、内存、运行时长硬限制，超出限制立即终止容器运行，返回对应异常状态。

3. **任务调度算法**：

- 判分任务队列采用 FIFO（先进先出）调度算法，保障任务执行顺序与提交顺序一致；
- 单用户提交限流算法：采用令牌桶限流，单用户每秒最多提交 1 次，1 分钟内最多提交 10 次，超出限制直接拦截。

4. **数据统计算法**：

- 用户学习数据采用离线定时统计算法，每日凌晨执行全量数据统计，更新用户核心指标，避免实时计算占用系统资源；
- 正确率、知识点掌握进度等指标采用加权统计算法，按题目难度设置不同权重，保障统计结果的合理性。

2.2.11 状态动态视角

本视角明确系统核心实体的状态定义、触发事件与状态流转规则，保障核心业务流程的状态一致性。

1. **代码提交记录状态机**：
   - 核心状态：待提交、预处理中、待判分、判分中、判分通过、判分不通过、编译失败、运行超时、内存超限、系统异常；
   - 初始状态：待提交；
   - 核心流转规则：用户提交代码→预处理中→校验通过进入待判分→判分引擎接管进入判分中→根据执行结果进入终态（判分通过/不通过/编译失败等）；
   - 终态说明：所有执行完成后的状态均为终态，不可逆向流转，用户重新提交代码生成新的提交记录，不修改原有记录状态。
2. **题目状态机**：
   - 核心状态：草稿、待审核、已上线、已下架、已删除；
   - 初始状态：草稿；
   - 核心流转规则：管理员新增题目→草稿→提交审核进入待审核→审核通过进入已上线→需修改/下架进入已下架→可重新编辑提交审核，或删除进入已删除；
   - 约束规则：仅已上线状态的题目可被普通用户查看与检索，其他状态题目仅管理员可操作。
3. **用户账号状态机**：
   - 核心状态：正常、临时锁定、禁用、已注销；
   - 初始状态：正常（注册成功后）；
   - 核心流转规则：连续 5 次登录失败→临时锁定（15 分钟后自动恢复为正常）；管理员违规处置→禁用（仅管理员可解除）；用户提交注销申请→已注销（终态，不可恢复）。

2.2.14 部署视角

本视角明确系统软件组件到物理节点的映射关系，以及部署架构与流程，保障系统可顺利部署运行。

1. **部署架构**：单服务器集中式部署，所有组件均部署于同一台 Linux 云服务器，具体映射关系如下：

软件组件	部署节点	部署方式	核心作用
Nginx 反向代理	云服务器	系统服务安装	静态资源服务、请求转发、HTTPS

			加密、限流
前端静态资源	云服务器	Nginx 目录部署	用户端与管理员端页面文件
SpringBoot 后端服务	云服务器	Jar 包部署，系统服务管理	核心业务服务、API 接口提供
MySQL 数据库	云服务器	系统服务安装	结构化数据存储
Docker 引擎	云服务器	系统服务安装	代码运行沙箱环境管理
Redis 缓存	云服务器	系统服务安装	缓存数据存储、限流、会话管理

2. **部署核心约束**:
   - 所有组件均采用 Docker 容器化部署或系统服务管理，开机自启，异常自动重启;
   - 部署目录权限严格管控，非必要组件禁止 root 权限运行;
   - 数据库开启定期自动备份，备份文件存储于独立目录，保障数据可恢复;
   - 服务器防火墙开启，仅开放必要端口，关闭所有非必要服务。

2.2.15 资源视角

本视角明确系统核心资源的使用、分配与管控规则，保障系统资源可控，避免资源滥用导致的系统不稳定。

1. **计算资源管控**:
   - 代码运行沙箱：单任务 CPU 核心限制为 1 核，内存限制为 256M，运行时长限制按题目要求设置，最长不超过 5 秒，超出限制立即终止;
   - 后端服务：JVM 堆内存限制为 2G，避免占用服务器全部内存;
   - 数据库：缓存池大小固定，避免数据库占用过多服务器资源。
2. **存储资源管控**:
   - 用户单文件代码限制不超过 10000 行，单文件大小不超过 1M;
   - 用户头像文件限制为 JPG/PNG 格式，大小不超过 2M;

- 系统日志、临时文件设置自动清理规则，保留 7 天内的日志文件，避免磁盘空间耗尽；
- 数据库单表数据量超过百万行时，自动分表处理，保障查询性能。

3. **接口调用资源管控**：
   - AI 功能调用：单用户每日代码评价最多 50 次、个性化错题解析最多 30 次、题目生成最多 20 道，单轮答疑会话最多 10 轮对话；
   - 邮件验证码：同一邮箱 1 分钟内最多发送 1 次验证码，1 小时内最多发送 5 次，验证码 15 分钟内有效；
   - 普通业务接口：单用户每秒请求数不超过 20 次，超出限制限流拦截。
4. **网络资源管控**：
   - 代码运行沙箱完全禁用外网访问，禁止任何网络连接；
   - 服务器外网带宽设置限流，避免单用户大流量下载占用全部带宽；
   - 所有外网请求均通过 HTTPS 加密传输，禁止 HTTP 明文传输。

3. 设计视图

3.1 整体架构设计

本系统采用**前后端分离的五层分层架构**，整体架构设计如下图核心逻辑所示，从上至下分为表现层、接入层、应用服务层、基础设施层、数据存储层，各层级职责单一，单向依赖，完全解耦。

```Plain Text
┌──────────────────────────────────────────────────────────────┐
│ 表现层：用户端 Web 页面 | 管理员后台页面 | 代码编辑器组件        │
└──────────────────────────────────────────────────────────────┘
                              │
                              ↓
┌──────────────────────────────────────────────────────────────┐
│ 接入层：Nginx 反向代理 | 请求限流 | 跨域处理 | HTTPS            │
└──────────────────────────────────────────────────────────────┘
                              │
                              ↓
┌──────────────────────────────────────────────────────────────┐
│ 应用服务层：SpringBoot 核心框架 | 六大业务模块 | 权限校验        │
│ 「用户管理 | 题库管理 | 自动判分 | AI 辅助 | 数据统计 | 后台管理」 │
└──────────────────────────────────────────────────────────────┘
                              │
                              ↓
```

```
┌──────────────────────────────────────────────────────────────┐
│ 基础设施层：OJ 判分引擎 | Docker 沙箱 | AI 服务客户端 |          │
│ 邮件服务 | 数据访问组件 | 缓存组件 | 任务调度组件               │
└──────────────────────────────────────────────────────────────┘
                              |
                              ↓
┌──────────────────────────────────────────────────────────────┐
│ 数据存储层：MySQL 数据库 | 本地文件存储 | Redis 缓存            │
└──────────────────────────────────────────────────────────────┘
```

架构核心优势：

1. 完全解耦：前后端分离、层级分离、模块分离，可并行开发、独立测试、灵活迭代；
2. 高可用：核心判分引擎与 AI 服务解耦，AI 故障不影响核心刷题流程，单点故障不导致系统整体不可用；
3. 高安全：多层安全防护，接入层限流、应用层权限校验、基础设施层沙箱隔离、存储层数据加密；
4. 可扩展：模块化设计，预留新增编程语言、新增 AI 模型、新增功能模块的扩展空间，无需修改核心架构；
5. 易落地：架构轻量化，匹配单服务器部署要求，适配项目团队技术能力与 12 周开发周期，无过度设计。

3.2 核心模块详细设计

3.2.1 用户管理模块

1. **模块职责**：负责用户账号全生命周期管理、身份认证、个人数据管理，是平台的基础用户体系模块。
2. **核心功能设计**：
    - 账号注册登录：基于 JWT 实现无状态身份认证，密码采用 BCrypt 不可逆加密存储；注册流程完成用户名/邮箱唯一性校验、邮箱验证码校验、密码强度校验；登录流程实现防暴力破解，连续 5 次失败临时锁定账号 15 分钟；登录状态有效期 7 天，超时需重新登录。
    - 个人信息管理：支持用户修改昵称、头像、个人简介、绑定邮箱，修改邮箱需重新完成验证码校验；头像文件上传后压缩存储，限制格式与大小。
    - 密码管理：支持原密码校验修改密码、邮箱验证码重置密码；密码修改成功后，原有登录状态自动失效。

- 提交记录管理：分页展示用户所有代码提交记录，支持按题目、提交时间、判分状态、编程语言筛选；普通用户仅可查看本人提交记录，不可查看他人非公开内容。

- 收藏与错题本管理：支持题目收藏/取消收藏、批量管理；系统自动收录用户未通过的题目至错题本，支持手动添加/移除错题、标记掌握状态，一键跳转重做错题。

- 账号注销：支持用户提交注销申请，完成身份校验后注销账号，按合规要求处理用户个人敏感数据，注销后数据不可恢复。

3. **模块接口设计**：提供注册、登录、密码重置、个人信息修改、提交记录查询、收藏/错题本管理、账号注销等标准化 RESTful 接口，所有接口除注册、登录外均需携带 JWT 令牌完成身份校验。

4. **异常处理**：所有用户操作均有明确的错误码与错误提示，敏感操作全流程日志记录，操作可溯源。

3.2.2 题库管理模块

1. **模块职责**：负责题目、分类、测试用例、官方题解的全生命周期管理，为前端提供题目检索、详情展示能力，是平台的核心内容模块。

2. **核心功能设计**：

- 题目分类管理：支持管理员对分类进行新增、编辑、删除、排序、启用/禁用操作；题目与分类为多对多绑定关系；已绑定题目的分类不可直接删除，禁用的分类不在前端展示。

- 题目基础信息管理：支持管理员完成题目的新增、编辑、上架、下架、删除操作；管理题目标题、描述、难度、分类、标签、时间/内存限制等核心属性；新增题目需填写所有必填字段，下架后的题目普通用户不可查看，题目修改后历史判分结果不回溯。

- 测试用例管理：支持管理员管理题目的公开样例用例与隐藏测试用例，支持单条增删改查与批量导入/导出；隐藏用例对普通用户完全不可见，仅返回执行结果；判分时所有测试用例全部执行，不可跳过。

- 官方题解管理：支持管理员编辑、发布、下架题目的官方标准解析与参考代码；官方题解需审核通过后，方可在前端展示。

- 题目检索与展示：前端分页展示已上线题目列表，支持按算法分类、难度、题目状态筛选，按关键词模糊搜索，按热度、发布时间排序；题目详情页完整展示题目信息、样例用例、用户历史提交记录。

3. **模块接口设计**：分为管理员后台接口与普通用户前端接口，管理员接口实现题目全生命周期管理，用户接口仅提供题目查询、详情查看能力，严格管控权限边界。

4. **数据约束**：题目 ID 为唯一主键，测试用例与题目为一对多关联关系，分类与题目为多对多关联关系，建立联合索引保障查询性能。

3.2.3 在线编程与自动判分模块

1. **模块职责**：负责在线代码编辑、代码提交预处理、自动判分核心逻辑、沙箱环境管理、判分结果推送，是平台的核心引擎模块。

2. **核心功能设计**：

    - 在线代码编辑器：基于 Monaco Editor 实现，支持 C++、Java、Python 三种核心编程语言，预留新增语言的扩展空间；提供代码高亮、自动缩进、代码格式化、全屏编辑、基础代码模板、快捷键支持；编辑器初始化时间不超过 1 秒，输入无明显延迟；自动保存代码草稿，刷新页面不丢失内容。

    - 代码提交与预处理：用户提交代码后，完成代码非空校验、行数限制校验、恶意代码静态检测；校验通过后生成唯一提交编号，将判分任务推入异步任务队列，采用异步判分模式；单用户提交设置限流规则，避免频繁提交占用资源。

    - 自动判分核心引擎：采用 Docker 容器化沙箱实现代码隔离运行，核心流程为：创建独立 Docker 容器→挂载代码与测试用例→编译代码（编译型语言）→编译失败直接返回错误信息→单条测试用例隔离运行→捕获运行结果、耗时、内存占用→对比标准输出→汇总所有用例执行结果→销毁容器与临时文件→返回最终判分报告。

    - 判分结果推送与展示：通过 WebSocket 实时向用户推送判分进度与最终结果；前端清晰展示最终状态、每个测试用例的执行情况、错误类型与详情、运行耗时与内存占用；编译错误、运行错误信息标注错误位置与原因，隐藏测试用例仅展示执行结果，不泄露用例内容。

3. **模块核心约束**：

    - 每次代码运行使用独立的 Docker 容器，运行结束后立即销毁，完全隔离平台系统；

    - 严格限制容器的 CPU、内存、运行时长，禁止外网访问、系统命令执行、危险函数调用；

    - 判分结果 100%准确，无错判、漏判，常规题目（≤10 个用例）从提交到返回结果最长不超过 8 秒，平均耗时不超过 3 秒。

4. **异常处理**：编译错误、运行超时、内存超限、输出错误、系统异常等场景均有明

确的状态码与错误提示，异常情况不导致系统崩溃，容器运行异常时立即销毁，避免资源泄露。

3.2.4 AI 智能辅助模块

1. **模块职责**：负责对接第三方 AI 大模型服务，实现 AI 代码评价、智能题解生成、个性化题目生成、编程答疑四大核心能力，同时完成 AI 生成内容的合规校验。
2. **核心功能设计**：
    - AI 代码多维评价：针对用户提交的代码，从运行效率、代码规范、可读性、健壮性四个维度完成全面评价，给出针对性优化建议与优化后的完整代码；支持手动触发，单用户每日最多调用 50 次；调用失败不影响核心判分流程。
    - AI 智能题解生成：支持生成题目的通用标准化 AI 题解，也可针对用户的错误代码生成个性化错题解析，精准定位错误原因，给出修复方案；通用题解需管理员审核通过后方可展示，题解中的参考代码需通过判分系统验证；单用户每日最多调用 30 次个性化错题解析。
    - AI 个性化题目生成：用户可自定义算法分类、难度、应用场景等参数，AI 生成完整的题目内容、题目描述、测试用例、题解代码；校验通过后纳入 AI 生成题库，用户可直接进入刷题流程；单用户每日最多生成 20 道题目，生成内容需经过合规校验，违规内容不展示。
    - AI 编程答疑：支持用户针对当前题目、代码、相关算法知识点发起对话式提问，AI 结合上下文完成实时解答；单轮会话最多支持 10 轮对话，仅回答与编程学习相关的问题，拒绝无关违规内容，对话全程合规审核。
3. **模块设计特点**：
    - 封装统一的 AI 服务客户端，适配多个 AI 大模型渠道，主渠道故障时可快速切换备用渠道，保障功能可用性；
    - 所有 AI 功能调用均设置频次限流，避免接口费用超支；
    - AI 生成内容全程合规校验，过滤违规、错误内容，避免错误引导；
    - 与核心业务流程解耦，AI 服务故障不影响平台核心功能使用。

3.2.5 数据统计模块

1. **模块职责**：负责用户个人学习数据与平台运营数据的统计、分析、可视化展示，为用户提供学习建议，为管理员提供运营决策支撑。
2. **核心功能设计**：

- 个人学习数据统计：统计用户刷题总数、正确率、知识点掌握进度、题目难度分布、学习打卡趋势、常见错误类型等核心数据；采用 ECharts 实现可视化展示，每日凌晨更新统计数据，基于用户学习情况给出针对性学习建议。
- 平台运营数据统计：管理员专属功能，统计平台用户注册数据、题库数据、判分系统运行数据、AI 功能使用数据、平台核心运行指标；可视化展示核心运营指标，异常指标高亮提示，不同权限管理员可查看的数据范围分级管控。

3. **模块设计特点**：
   - 采用离线定时任务统计核心指标，避免实时计算占用系统资源，影响核心业务性能；
   - 统计数据与用户实际操作完全一致，无统计错误；
   - 严格管控数据访问权限，普通用户仅可查看本人数据，管理员按权限查看运营数据。

3.2.6 后台管理模块

1. **模块职责**：负责平台用户与权限管理、AI 内容审核、平台配置、运行状态监控，为管理员提供完整的平台运营管理能力。
2. **核心功能设计**：
   - 用户与权限管理：基于 RBAC 模型实现角色与权限管控，支持管理员查看全平台用户列表，启用/禁用用户账号，分配用户角色与权限；仅超级管理员可分配管理员权限，权限修改后实时生效，原有登录状态自动失效；支持查看用户操作日志，敏感操作可溯源。
   - AI 内容审核管理：支持管理员查看所有 AI 生成的题目与解析内容，完成内容审核、编辑、纳入官方题库、下架/删除操作；未通过审核的内容不对普通用户展示，违规内容可一键处置。
   - 平台配置与监控：支持超级管理员配置平台基础参数，查看平台运行状态、接口响应情况、判分系统运行状态、AI 接口调用情况；查看异常日志与告警信息，异常问题可快速定位。
3. **模块核心约束**：
   - 所有后台操作均需完成双重权限校验，越权操作 100%拦截；
   - 所有管理员操作全流程日志记录，操作人、操作时间、操作内容、IP 地址全程可溯源；
   - 平台核心配置参数修改需二次校验，避免误操作导致系统故障。

3.3 数据库设计

3.3.1 数据库选型与设计原则

1. **数据库选型**：采用 MySQL 5.7 关系型数据库，支持事务、行级锁，保障数据一致性与并发性能，适配项目技术栈与部署环境。
2. **核心设计原则**：
    - 遵循数据库三大范式，减少数据冗余，避免数据不一致；
    - 所有表均设置主键、创建时间、更新时间字段，核心数据设置逻辑删除字段，不做物理删除；
    - 针对高频查询字段建立索引，避免全表扫描，保障查询性能；
    - 敏感字段加密存储，用户隐私数据脱敏处理，保障数据安全；
    - 表名、字段名采用统一命名规范，注释完整，便于维护。

3.3.2 核心数据表设计

表名	核心用途	关键字段	索引设计
sys_user	用户信息表	user_id(主键)、username、email、password(加密)、nickname、avatar、status、create_time、update_time	用户名唯一索引、邮箱唯一索引、用户状态普通索引
sys_role	角色表	role_id(主键)、role_name、role_key、status、create_time	角色标识唯一索引
sys_user_role	用户角色关联表	id(主键)、user_id、role_id	user_id+role_id 联合唯一索引

sys_menu	权限菜单表	menu_id(主键)、parent_id、menu_name、perms、type、order_num	父菜单 ID 普通索引
sys_role_menu	角色权限关联表	id(主键)、role_id、menu_id	role_id+menu_id 联合唯一索引
problem_category	题目分类表	category_id(主键)、category_name、sort、status、create_time	分类名称唯一索引、状态普通索引
problem_info	题目信息表	problem_id(主键)、title、description、difficulty、time_limit、memory_limit、status、create_time	题目标题索引、难度索引、状态索引、创建时间索引
problem_category_rel	题目分类关联表	id(主键)、problem_id、category_id	problem_id+category_id 联合唯一索引
problem_test_case	题目测试用例表	case_id(主键)、problem_id、input、output、is_public、sort	problem_id 普通索引、是否公开普通索引
problem_solution	题目官方题解表	solution_id(主键)、problem_id、content、reference_code、status、create_time	problem_id 唯一索引、状态普通索引

code_submission	代码提交记录表	submit_id(主键)、user_id、problem_id、code、language、status、run_time、run_memory、create_time	user_id 普通索引、problem_id 普通索引、提交状态普通索引、创建时间索引
user_collection	用户收藏表	id(主键)、user_id、problem_id、create_time	user_id+problem_id 联合唯一索引
user_error_book	用户错题本表	id(主键)、user_id、problem_id、master_status、create_time	user_id 普通索引、problem_id 普通索引、掌握状态普通索引
ai_generated_content	AI 生成内容表	content_id(主键)、user_id、content_type、content、problem_id、status、create_time	user_id 普通索引、内容类型普通索引、状态普通索引
sys_operation_log	操作日志表	log_id(主键)、user_id、operation_type、content、ip、create_time	user_id 普通索引、操作类型普通索引、创建时间索引

3.3.3 数据一致性与备份设计

1. **数据一致性保障**：核心业务操作均采用数据库事务保障原子性，如用户注册、题目与测试用例同步修改、判分结果保存，避免部分成功部分失败导致的数据不一致；高频更新数据采用乐观锁，避免并发更新导致的数据覆盖。

2. **数据备份方案**：数据库开启每日全量自动备份，备份文件保留不少于 30 天；备份文件存储于独立目录，支持一键恢复；核心数据修改前自动备份，避免误操作导致的数据丢失。

3.4 接口详细设计

3.4.1 接口通用规范

1. **通信协议**：常规业务接口采用 HTTPS RESTful 协议，实时推送接口采用 WebSocket 协议；
2. **请求方法**：查询操作使用 GET，新增操作使用 POST，修改操作使用 PUT，删除操作使用 DELETE；
3. **统一响应格式**：所有接口返回统一的 JSON 格式，结构如下：

```JSON
{
  "code": 200, // 状态码：200 成功，其他为失败
  "msg": "操作成功", // 提示信息
  "data": {} // 响应数据，无数据时为 null
}
```

4. **权限规范**：除注册、登录、题目浏览等公开接口外，所有接口请求头需携带 `Authorization: Bearer {JWT 令牌}`，后端完成身份与权限校验，校验失败返回 401/403 状态码；
5. **异常规范**：所有异常均捕获并返回清晰的错误码与错误信息，无未捕获异常导致的服务崩溃；客户端错误返回 4xx 状态码，服务端错误返回 5xx 状态码。

3.4.2 核心接口示例

1. **用户登录接口**
   - 接口地址：`/api/user/login`
   - 请求方法：POST
   - 请求参数：

   ```JSON
   {
     "username": "string", // 用户名/邮箱
     "password": "string" // 密码
   ```

```
}
```

- 响应数据:

```JSON
{
  "code": 200,
  "msg": "登录成功",
  "data": {
    "token": "string", // JWT 令牌
    "userInfo": {
      "userId": "number",
      "username": "string",
      "nickname": "string",
      "avatar": "string",
      "roles": ["string"]
    }
  }
}
```

2. **题目列表查询接口**

- 接口地址: `/api/problem/list`
- 请求方法: GET
- 请求参数:

参数名	类型	必填	说明
pageNum	int	是	页码
pageSize	int	是	每页条数
keyword	string	否	搜索关键词
categoryId	int	否	分类 ID
difficulty	int	否	难度: 1 简单 2 中等 3 困难
sort	string	否	排序方式: hot/time

- 响应数据：分页题目列表，包含题目 ID、标题、难度、分类、通过率、创建时间等信息。

3. **代码提交接口**
   - 接口地址：`/api/judge/submit`
   - 请求方法：POST
   - 请求头：`Authorization: Bearer {JWT 令牌}`
   - 请求参数：

```JSON
{
  "problemId": "number", // 题目 ID
  "code": "string", // 代码内容
  "language": "string" // 编程语言：cpp/java/python
}
```

   - 响应数据：提交编号、提交状态，前端通过 WebSocket 监听判分进度与结果。

4. **AI 代码评价接口**
   - 接口地址：`/api/ai/code-review`
   - 请求方法：POST
   - 请求头：`Authorization: Bearer {JWT 令牌}`
   - 请求参数：

```JSON
{
  "submitId": "number" // 提交记录 ID
}
```

   - 响应数据：AI 多维度评价内容、优化建议、优化后代码。

3.4.3 WebSocket 接口规范

1. **连接地址**：`wss://{域名}/ws/{userId}`，连接时需携带 JWT 令牌完成身份校验，校验失败拒绝连接；
2. **心跳机制**：客户端每 30 秒发送一次心跳包，服务端 60 秒未收到心跳包自动断开连接；

3. **消息格式**：统一 JSON 格式，包含消息类型、数据体、时间戳；
4. **核心消息类型**：判分进度推送、判分结果推送、AI 生成进度推送、AI 结果推送、异常告警推送。

3.5 安全设计

本系统从账号安全、接口安全、代码运行安全、数据安全、内容安全五个维度构建全链路安全防护体系，完全对齐需求分析文档的安全要求。

3.5.1 账号与数据安全

1. 用户密码采用 BCrypt 不可逆加密算法存储，不保存明文密码；
2. 账号具备防暴力破解机制，连续 5 次登录失败临时锁定 15 分钟；
3. 用户敏感信息（邮箱、手机号）脱敏展示与存储，避免信息泄露；
4. 密码修改、账号注销等敏感操作需二次身份校验；
5. 平台数据每日自动备份，备份数据保留不少于 30 天，保障数据可恢复；
6. 采用 HTTPS 协议传输所有数据，避免明文传输导致的数据泄露。

3.5.3 代码运行安全

1. 采用 Docker 容器构建独立沙箱环境，每次代码运行创建新容器，运行结束后立即销毁，完全隔离平台系统；
2. 沙箱严格限制 CPU、内存、运行时长，超出限制立即终止运行，避免资源耗尽；
3. 沙箱禁用外网访问、禁止系统命令执行、禁止 root 权限、禁止宿主机目录挂载，全程监控系统调用；
4. 代码提交前进行静态恶意代码检测，检测到危险代码直接禁止提交；
5. 代码运行时全程监控，异常行为立即终止容器，返回对应异常信息，杜绝恶意代码攻击风险。

3.5.4 内容安全

1. AI 生成内容、用户提交内容全程进行合规校验，过滤违规、色情、暴力、政治敏感内容；
2. 未通过审核的 AI 生成内容不对普通用户展示，管理员可一键处置违规内容；
3. 平台内容与用户数据处理完全符合国家相关法律法规要求，无违规内容存储与传播。

3.6 性能优化设计

本系统针对需求分析文档中的性能指标要求，从前端、后端、数据库、判分引擎四个维度进行全链路性能优化设计，保障所有性能指标达标。

3.6.1 前端性能优化

1. 页面首屏加载优化：采用路由懒加载、静态资源压缩、CDN 加速，首屏加载时间不超过 1.5 秒；
2. 页面切换优化：采用前端缓存、组件复用，页面切换响应时间不超过 0.5 秒；
3. 代码编辑器优化：采用 Monaco Editor 按需加载，初始化时间不超过 1 秒，输入响应无明显延迟；
4. 列表渲染优化：采用虚拟列表处理长列表数据，避免页面卡顿，滚动流畅无异常；
5. 浏览器兼容性：适配 Chrome、Edge、Firefox、Safari 主流浏览器，支持 1366×768 至 4K 全分辨率自适应。

3.6.2 后端性能优化

1. 接口性能优化：常规查询操作平均响应时间不超过 300 毫秒，最长不超过 1 秒；数据写入操作平均响应时间不超过 500 毫秒，最长不超过 2 秒；
2. 异步处理优化：判分、AI 调用、数据统计等耗时操作采用异步任务队列处理，不阻塞主线程；
3. 缓存优化：高频访问的题目数据、分类数据、用户基础信息存入 Redis 缓存，减少数据库查询压力；
4. 限流优化：采用令牌桶算法实现接口限流，避免高并发请求导致系统崩溃，保障系统在 200 人并发操作时无报错、无响应超时；
5. 资源占用优化：JVM 参数调优，固定堆内存大小，避免频繁 GC 导致的性能下降；Docker 沙箱资源硬限制，避免单任务占用过多资源。

3.6.3 数据库性能优化

1. 索引优化：针对高频查询字段建立合适的索引，避免全表扫描，单条数据查询响应时间不超过 200 毫秒，多条件联合查询响应时间不超过 500 毫秒；
2. 分表优化：代码提交记录表等高频写入的表，按时间分表处理，支持百万级数据存储，查询性能无明显下降；
3. 事务优化：精简数据库事务范围，避免长事务导致的锁等待；高频更新数据采用乐观锁，减少锁竞争；

4. 连接池优化：配置合理的数据库连接池参数，避免连接数不足导致的请求阻塞，保障高并发下的数据库稳定性。

3.6.4 判分引擎性能优化

1. 任务调度优化：采用固定线程池处理判分任务，避免频繁创建销毁线程导致的性能损耗；
2. 沙箱优化：Docker 镜像精简优化，容器启动时间控制在 500 毫秒内；
3. 用例执行优化：测试用例预加载，避免重复读取文件；单条用例执行结果实时推送，无需等待全量用例执行完成；
4. 性能指标保障：常规题目（≤10 个用例）从提交到返回最终结果最长不超过 8 秒，平均耗时不超过 3 秒；支持最多 1000 个判分任务同时排队，无任务丢失、无顺序错乱；判分结果准确率 100%。

4. 关键设计决策

本章节明确项目核心架构、技术选型、实现方案的关键决策，以及每个决策的选型依据、备选方案与决策理由，所有决策均匹配项目需求、团队能力与开发周期要求。

决策编号	设计决策内容	备选方案	决策依据与理由
D-001	整体采用前后端分离的 B/S 五层分层架构	传统 MVC 一体化架构、微服务架构	1. 前后端分离架构可实现前后端并行开发，提升开发效率，适配 6 人团队的协作模式；2. 分层架构职责清晰，代码可维护性强，符合毕业设计项目的评审要求；3. 微服务架构过于复杂，超出项目范围与开发周期，单服务器部署也无需微服务拆分；4. 传统 MVC 架构前后端耦合度高，迭代效率低，不符合当前技术主流规范
D-002	采用 Docker 容器化沙箱实现代码运行与自动判分	虚拟机方案、Linux Namespace 原生隔离方案、直接在宿主机运行	1. Docker 容器隔离性强，可实现运行环境与宿主机完全隔离，杜绝恶意代码攻击风险，满足系统安全要求；2. Docker 容器启动速度快，资

			源占用低，可满足判分引擎的性能要求；3. 虚拟机方案资源占用高、启动慢，无法满足快速判分的需求；4. 原生隔离方案开发难度大，适配性差，超出团队技术能力与开发周期；5. 直接在宿主机运行存在极大的安全风险，完全不可行
D-003	判分流程采用异步任务队列+WebSocket 实时推送方案	同步等待判分结果方案、轮询查询判分状态方案	1. 异步队列可削峰填谷，应对高并发提交场景，保障系统稳定性，支持 1000 个任务排队；2. WebSocket 全双工通信可实时推送判分进度，用户体验远优于轮询方案，同时减少服务器压力；3. 同步等待方案会导致请求超时，高并发下会耗尽服务器线程资源，导致系统崩溃；4. 轮询方案频繁请求服务器，资源浪费严重，且实时性差，用户体验不佳
D-004	AI 智能辅助模块与核心业务流程解耦设计	AI 功能与核心业务强耦合设计	1. 解耦设计可保障 AI 服务故障时，用户注册登录、代码编写、自动判分等核心功能可正常使用，提升系统可用性；2. 解耦设计便于后续更换 AI 模型、新增 AI 功能，无需修改核心业务代码，提升系统可扩展性；3. 强耦合设计会导致 AI 服务故障时整个系统不可用，风险极高，不符合系统稳定性要求
D-005	后台权限管理采用 RBAC 基于角色的访问控制模	直接给用户分配权限的 ACL 模型	1. RBAC 模型实现了用户与权限的解耦，通过角色灵活分配权限，适配平台普通管

	型		理员、超级管理员等不同角色的权限管控需求；2. 当管理员数量增加、权限变更时，RBAC 模型的管理效率远高于 ACL 模型，可维护性更强；3. ACL 模型仅适合简单的权限场景，无法满足平台多角色、多权限的管理需求
D-006	采用单服务器集中式部署架构	多服务器分布式部署架构	1. 单服务器部署符合项目需求文档中“暂不包含多服务器分布式部署”的范围要求；2. 单服务器部署成本低、运维简单，匹配项目预算与学生项目的实际情况；3. 架构设计预留了分布式扩展的空间，后续可平滑升级为分布式部署；4. 多服务器分布式部署超出项目范围、预算与运维能力，无实际必要性
D-007	后端技术选型：SpringBoot 2.7.x + MySQL 5.7 + Redis	SSM 框架、Python Django/Flask 框架、Node.js Express 框架	1. SpringBoot 是 Java 企业级开发的主流框架，文档丰富、生态完善，团队成员具备相关开发经验，可快速落地；2. MySQL 是主流的关系型数据库，适配项目结构化数据存储需求，团队成员熟悉相关操作；3. Redis 用于缓存与限流，提升系统性能，技术成熟稳定；4. SSM 框架配置繁琐，开发效率低于 SpringBoot；Python/Node.js 框架团队成员熟练度不足，存在技术风险，且不利于 OJ 引擎的集成

D-008	前端技术选型：Vue3 + Vite + Element Plus + Monaco Editor	React 框架、Angular 框架	1. Vue3 上手难度低，生态完善，团队成员具备相关开发经验，可快速实现页面与交互逻辑；2. Vite 构建速度快，开发体验好，可提升开发效率；3. Element Plus 提供了完善的后台组件库，适配管理员后台开发需求；4. Monaco Editor 是 VS Code 的底层编辑器，功能完善，适配在线代码编辑需求；5. React/Angular 框架团队成员熟练度不足，存在技术风险，开发周期不可控
D-009	数据统计采用离线定时任务方案	实时计算方案	1. 离线定时任务在凌晨低峰期执行统计，不占用白天业务高峰期的系统资源，保障核心业务性能；2. 学习数据、运营数据无需实时更新，每日更新完全满足用户需求；3. 实时计算会占用大量数据库与服务器资源，高并发下会影响核心业务性能，无实际必要性

5. 附录

5.2 开发规范

1. 《Java 后端代码开发规范》
2. 《Vue 前端代码开发规范》
3. 《MySQL 数据库开发规范》
4. 《Git 版本控制与分支管理规范》
5. 《接口设计与开发规范》

5.3 部署环境清单

1. 服务器环境：Linux CentOS 7.9 2 核 4G 100G 硬盘 5M 带宽
2. 运行环境：JDK 1.8、Maven 3.6、MySQL 5.7、Redis 6.2、Docker 24.0、Nginx 1.24
3. 开发环境：Windows 10+、IntelliJ IDEA、VS Code、CLion、Postman

5.4 异常错误码清单

错误码	错误类型	说明
200	成功	操作成功
400	参数错误	请求参数校验失败
401	未授权	未登录或登录状态失效
403	权限不足	无对应操作权限
404	资源不存在	请求的资源不存在
429	请求限流	请求过于频繁，触发限流
500	服务端错误	服务器内部异常
1001	账号错误	用户名或密码错误
1002	账号锁定	账号已被临时锁定或禁用
2001	题目不存在	题目不存在或已下架
3001	代码提交失败	代码校验失败或包含恶意代码
3002	判分异常	判分引擎执行异常
4001	AI 调用失败	AI 服务调用异常
4002	AI 调用超限	AI 功能调用次数超出当日

		限制

|（注：文档部分内容可能由 AI 生成)